

B-Minor 2025 Language Overview

Note: The B-Minor language used in this class changes each year in order to provide new challenges and opportunities. This document differs from the one in the textbook in the following ways:

- Integers may be represented as decimal, hexadecimal or binary.
- Strings and characters have a number of additional escape codes.
- Double precision floating point values, types, and operators have been added.
- Arrays have an intrinsic length that is bounds-checked at runtime and read by the `#` operator.
- `auto` indicates a variable type to be inferred from context.

Overview

This is an informal specification of B-Minor 2025, a C-like language for use in an undergraduate compilers class. B-minor includes expressions, basic control flow, recursive functions, and strict type checking. It is object-code compatible with ordinary C and thus can take advantage of the standard C library, within its defined types. It is similar enough to C to feel familiar, but different enough to give you some sense of alternatives.

This document is deliberately informal: the most precise specification of a language is the compiler itself, and it is your job to write the compiler! It is your job to read the document carefully and extract a formal specification. You will certainly find elements that are unclear or incompletely specified, and you are encouraged to raise questions in class.

Whitespace and Comments

In B-minor, whitespace is any combination of the following characters: tabs, spaces, linefeed (`\n`), and carriage return (`\r`). The placement of whitespace is not significant in B-minor. Both C-style and C++-style comments are valid in B-minor:

```
/* A C-style comment */  
a=5; // A C++ style comment
```

Identifiers and Keywords

Identifiers (i.e. variable and function names) may contain letters, numbers, and underscores. Identifiers must begin with a letter or an underscore and may be up to 255 characters long. These are examples of valid identifiers:

```
i x mystr fog123 BigLongName55
```

The following strings are B-minor keywords (or reserved for future expansion) and may not be used as identifiers:

```
array auto boolean carray char else false float double for function if integer print return string true void while
```

Types

B-minor has five atomic types: integers, doubles, booleans, characters, and strings. A variable is declared as a name followed by a colon, then a type and an optional initializer. For example:

```
x: integer;  
y: integer = 123;  
f: double = 45.67;  
b: boolean = false;  
c: char    = 'q';  
s: string  = "hello bminor\n";
```

- `integer` is a signed 64-bit value. An integer literal can be represented as a decimal (123), a hexadecimal value with an `'H'` suffix (`'104A3F1H'`), or a binary value with a `'B'` suffix (`'0101110101B'`), a hexadecimal value like `0x104A3F`, or a binary value like `0b0101110101`. All three forms may omit leading zeroes as needed.
- `double` is a floating point number that follows the [IEEE 754 double-precision standard](#). `double` values can be represented in various ways. One method is to place the integer component, a period, and the fractional component all in a row, such as in `12.34`. Another valid method is to use the scientific representation of the value, which involves having a floating point value as described above followed by an exponent (represented as either `e` or `E`). The exponent value may be preceded by an optional plus or minus sign. Some examples of this would be `5.67E1` which has the value of 56.7, or `89e-2` which has the value of .89.
- `boolean` can take the literal values `true` or `false`.
- `char` is a single 8-bit ASCII character enclosed in single quotes.
- `string` is a double-quoted constant string of ASCII characters that is immutable and (internally) null-terminated. Strings may contain up to 255 characters. (Backslash codes noted below each indicate one character.)

Both `char` and `string` literals may contain any ASCII value between 32 and 127. Values outside that range can be represented by the following backslash codes:

Code	Value	Meaning
<code>\a</code>	7	Bell
<code>\b</code>	8	Backspace
<code>\e</code>	27	Escape
<code>\f</code>	12	Form Feed (clear)
<code>\n</code>	10	Newline
<code>\r</code>	13	Carriage Return
<code>\t</code>	9	Tab
<code>\v</code>	11	Vertical Tab
<code>\\</code>	92	Backslash
<code>\'</code>	39	Single Quote
<code>\"</code>	34	Double Quote
<code>\0xHH</code>	HH	The byte value given by two characters HH in hexadecimal.

Any *other* character following a backslash represents exactly that character.

Arrays

B-minor supports local and global arrays of a fixed size. They may be declared with no value, which causes them to contain all zeros:

```
a: array [5] integer;
```

Or, the entire array may be given specific values:

```
a: array [5] integer = {1,2,3,4,5};
```

The length of an array is given by the `#` operator:

```
l: integer = # a;
```

Array accesses are bounds-checked at runtime. If a program attempts to access an illegal array element, the program will be aborted with a suitable error message, thus avoiding undefined behavior. For example, the following program will attempt to run off the end of the array, and be aborted at runtime:

```
for( i=0; i<=#a; i++ ) {
    print a[i];
}
```

To allow for compatibility with C and other languages that do not perform bounds checking, the type `carray` provides simple arrays that do **not** support the `#` operator, do not perform bounds checking, and are therefore unsafe. For example, this example runs off the end of the array and will have undefined behavior:

```
c: carray [5] integer = {1,2,3,4,5};
for( i=0; i<=5; i++ ) {
    print c[i];
}
```

Expressions

B-minor has many of the arithmetic operators found in C, along with a few additions, following a similar order of precedence:

Symbol	Meaning
() [] f()	grouping, array subscript, function call
++ --	postfix increment, decrement
#	unary array length
- !	unary negation, logical not
^	exponentiation
* / %	multiplication, division, remainder
+ -	addition, subtraction
< <= > >= == !=	comparison
&&	logical and
	logical or
=	assignment

B-minor is **strictly typed**. This means that you may only assign a value to a variable (or function parameter) if the types match **exactly**. You cannot perform many of the fast-and-loose conversions found in C.

Following are examples of some (but not all) type errors:

```
x: integer = 65;
y: char = 'A';
if(x>y) ... // error: x and y are of different types!

f: integer = 0;
if(f) ... // error: f is not a boolean!

writechar: function void ( c: char );
a: integer = 65;
writechar(a); // error: a is not a char!

b: array [2] boolean = {true,false};
x: integer = 0;
x = b[0]; // error: x is not a boolean!
```

```
x: integer = 0;
y: double = x; // error: integer types can not be implicitly converted to double
```

Following are some (but not all) examples of correct type assignments:

```
b: boolean;
x: integer = 3;
y: integer = 5;
b = x<y;      // ok: the expression x<y is boolean

f: integer = 0;
if(f==0) ...  // ok: f==0 is a boolean expression

c: char = 'a';
if(c=='a') ... // ok: c and 'a' are both chars
```

The `auto` type is used to declare a variable whose (fixed) type is to be inferred at compile-time:

```
s: auto = "hello"; // s will have type string

t: auto = x < y;    // t will have type boolean

f: function auto ( x: integer, y: integer ) =
{
    return x + y; // f will have return type integer
}
```

Declarations and Statements

In B-minor, you may declare global variables with optional constant initializers, function prototypes, and function definitions. Within functions, you may declare local variables (including arrays) with optional initialization expressions. Scoping rules are identical to C. Function definitions may not be nested.

Within functions, basic statements may be arithmetic expressions, return statements, print statements, if and if-else statements, for loops, or code within `inner{}` groups:

```
// An arithmetic expression statement.
y = m*x + b;

// An if-else statement.
if( temp>100 ) {
    print "It's really hot!\n";
} else if( temp>70 ) {
    print "It's pretty warm.\n";
} else {
    print "It's not too bad.\n";
}

// A for loop statement.
for( i=0; i<100; i++ ) {
    print i;
}

// A return statement.
return (f-32)*5/9;
```

B-minor does not have switch statements, while-loops, or do-while loops. (But you could consider adding them as a little extra project.)

The `print` statement is a little unusual because it is a **statement** and not a function call like `printf` is in C. `print` takes a list of expressions separated by commas, and prints each out to the console, like this:

```
print "The temperature is: ", temp, " degrees\n";
```

Note that each expression in the list may have a distinct type. `print` will determine the proper type and print accordingly.

Functions

Functions are declared in the same way as variables, except giving a type of `function` followed by the return type, arguments, and code:

```
square: function integer ( x: integer ) = {  
    return x^2;  
}
```

The return type must be one of the four atomic types, `auto` to indicate inferred type, or `void` to indicate no return value. Function arguments may be of any type except `auto` or `void`. `integer`, `boolean`, and `char` arguments are passed by value, while `string` and `array` arguments are passed by reference. When used as a function parameter, the length of an array is omitted, and can be obtained at runtime with the `#` operator.

```
printarray: function void ( a: array [] integer ) = {  
    i: integer;  
    for( i=0; i<#a; i++) {  
        print a[i], "\n";  
    }  
}
```

A function prototype may be given, which states the existence and type of the function, but includes no code. This must be done if the user wishes to call an external function linked by another library. For example, to invoke the C function `puts`:

```
puts: function void ( s: string );  
  
main: function integer () = {  
    puts("hello world");  
}
```

A complete program must have a `main` function that returns an integer. The arguments to `main` have the same meaning as `argc` and `argv` in the C language: `argc` indicates the number of arguments, and `argv` is an (unsafe) array of strings:

```
main: function integer ( argc: integer, argv: carray [] string ) = {  
    puts("hello world");  
}
```

Standards Committee Clarifications

Working out all the corner cases of a language from a specification is a surprising amount of work. Even a good informal statement of a language can leave a number of details uncertain. At various points in the semester, we will hold “standards committee” meetings to discuss and vote on any ambiguous aspects of the specification.

Scanning (1.X)

- **Q1.0: Is "" a valid string literal?**

A: Yes, two double quotes represents an empty string consisting only of the null terminator.

- **Q1.1: Is this a valid string literal?**

```
"hello  
world"
```

A: No, a newline in a string needs to be escaped, like this: `"hello\nworld"`

- **Q1.2: Do we need to handle `#include`, `#define`, and so forth?**

A: No, they are not part of B-minor.

- **Q1.3: Can an integer have a leading negative/positive sign?**

A: Yes, it can have a leading sign. However, we recommend that you treat this as two separate tokens like `TOKEN_MINUS TOKEN_INTEGER` so as to avoid some parsing problems later on.

- **Q1.4: Can a hexadecimal constant use either upper or lower case characters? (like `af35h` or `F6AC92H` or `1234deB5H`?)**

A: Decided: Hexadecimal numbers may use either upper or lower case. (Sep 22, 2025)

- **Q1.5: Should a token like `A123H` be treated as an identifier or a hexadecimal constant?**

A: Decided: Hexadecimal numbers must be represented as `0x1234abcd`. The `x` must be lower case, while the digits may be either case. Likewise, binary numbers must be represented as `0b010101`, and the `b` must be lower case. (These were changed to be more consistent with escape codes.) (Sep 22, 2025)

- **Q1.7: Can a string contain an unescaped quote, like `""`?**

A: Decided: A quotation mark within a string must be escaped, like `"\"` (Sep 22, 2025)

Parsing (2.X)

- **Q2.0: Is `print`; a valid statement?**

A: Yes, it means to print out nothing.

- **Q2.1: Is `return`; a valid statement?**

A: Yes, it indicates a return with no value in a `void` function.

- **Q2.2: Does B-minor permit this syntax?**

```
for(i=0;i<10,j<10;i++) { ... }
```

A: No, commas may only be used in `print` statements, function calls, function prototypes, and array expressions.

- **Q2.3: Can a single statement (without braces) be used after a `for`-loop or an `if`-statement?**

A: Yes, the following are valid statements, just as in C and C++:

```
for(i=0;i<10;i++) print i;  
if(a) x=y; else z=w;
```

- **Q2.4: Is a single semicolon a valid statement?**

A: No.

- **Q2.5: Can an array be zero length?**

A: No - An array must be declared with a positive length.

- **Q2.6: Can an array initializer be empty?**

A: No - An initializer must either match the length of the array, or be omitted. It cannot be empty. (It also avoids the case of an empty initializer `{}` begin confused with an empty statement block `{}`).

- **Q2.7: Is an empty file (no declarations) a valid B-minor module?**

A: Yes - An empty file should parse, typecheck, and compile to an empty assembly program. However, it won't be a complete linkable program without a `main` function.

Name Resolution and Type Checking

- **Q3.0: Can a declaration be used as the sole statement within an if or for statement?**

A: No - The following statements are invalid:

```
if(x<10) a: integer = 5;  
else    b: integer = 10;  
  
for(i=0;i<10;i++) c: integer = 30;
```

You may detect this problem at any stage of the compiler where it is most convenient: parsing, name resolution, or typechecking.